Schichtplan Manager
11. November 2026

Prof. Dr. Christina Büsing
Linus Palm
palm@combi.rwth-aachen.de

Lehr- und Forschungsgebiet
Kombinatorische Optimierung

Modell

Dieses Dokument beschreibt das gemischt-ganzzahlige lineare Programm (MILP), mit dem der Gurobi-Solver (`api/app/solver/milp/service.py`, Klasse `GurobiSolverService`) den Dienstplan eines Monats erstellt. Eine Fassung ohne Formeln ist die *Modell – Kurzfassung*.

1 Überblick

Was entschieden wird. Für jede Person, jeden Tag des Monats und jede Schicht: arbeitet sie diese Schicht und auf welcher Station? Das Modell ist allgemein für mehrere Stationen formuliert; die Demo plant genau eine, die fiktive Musterstation (`api/app/demo.py`).

Feste Regeln und Ziele. Jeder Plan hält die festen Regeln aus Abschnitt 6 ein. Alles andere ist ein Ziel. Die elf Ziele werden lexikografisch optimiert (Abschnitt 8), also nacheinander in dieser Rangfolge, wobei ein späteres Ziel ein früheres nicht verschlechtern darf:

Rang	Ziel	Code	Abschnitt
1	Monatssaldo innerhalb ±5 h	`balance_violation`	7.1
2	Wochenende über den Monatswechsel fortführen	`weekend_start`	7.2
3	möglichst wenige Lücken	`gaps`	7.3
4	Lücken gleichmäßig auf die Tage verteilen	`gap_spread`	7.4
5	Tage gleichmäßig besetzen	`staffing_spread`	7.5
6	Samstage je Qualifikation gleichmäßig verteilen	`saturdays`	7.6
7	Wünsche erfüllen, je Person gleich gewichtet	`wishes`	7.7
8	Rotation: Früh, Spät, Nacht in jedem 28-Tage-Fenster	`rotation`	7.8
9	Schichten eines Tages gleichmäßig besetzen	`shift_balance`	7.9
10	möglichst wenige Wechsel Spät → Früh	`Min_L_to_E`	7.10
11	möglichst oft dieselbe Schicht wie am Vortag	`cons`	7.11

Monatswechsel. Zwei Dinge verbinden aufeinanderfolgende Monate: Mit geladenem Vormonat zählen seine Dienste als feste Werte bei allen Regeln mit. Endet ein Monat an einem Samstag, wird zudem der folgende Sonntag mitgeplant (Abschnitt 6), damit jeder Samstagsdienst im nächsten Monat weitergehen kann.

Lesehilfe. Die Tage werden fortlaufend nummeriert: 1 ist der erste Tag des Monats, n der letzte, 0 der letzte Tag davor. Dienste vor dem Monat sind feste Werte, keine Variablen. Jeder Abschnitt gibt zuerst die Formeln und darunter die Erklärung mit Verweis auf die Gleichungsnummern; in Klammern steht der Name im Code.

2 Mengen und Indizes

$i \in I$	Mitarbeitende	(1)
$u \in U, \quad U^{St} \subseteq U$	Planungseinheiten, davon Stationen	(2)
$s \in S$	Schichten	(3)
$\theta \in \Theta = \{\text{Früh}, \text{Zwischen}, \text{Spät}, \text{Nacht}\}, \quad S^{\theta} \subseteq S$	Schichtarten und ihre Schichten	(4)
$l \in L = \{\text{Fachkraft}, \text{Hilfskraft}, \text{Azubi}, \text{MFA}\}$	Qualifikationen	(5)
$d \in D = \{1, \ldots, n\}, \quad D^{+} = D \cup \{n+1\}$	Tage des Monats, dazu der Folgetag	(6)
$\hat{D} = \{1, \ldots, \hat{n}\}$	Planungstage: der Monat, ggf. mit Sonntag	(7)
$d \in D^{-} = \{1-m, \ldots, 0\}, \quad \bar{D} = D^{-} \cup D$	Vortage mit bekannten Diensten, Planungshorizont	(8)
$D^{Sa} \subseteq \{0, \ldots, \hat{n}-1\}$	Samstage, deren Sonntag geplant wird	(9)
$W = (w_1, \ldots, w_{\lvert W \rvert}) \subseteq \{-m, \ldots, n\}$	Samstage der Wochenenden mit einem Tag in $\bar{D}$	(10)
$D^{end} = \{1 - \max(\Delta^N, L^N + 1), \ldots, \hat{n}-1\}$	Tage, an denen ein Nachtblock enden kann	(11)
$K \subseteq S \times D \times U^{St} \times L$	Bedarfszeilen mit $r_{sdul} > 0$	(12)
$A \subseteq I \times S \times \hat{D} \times U^{St}$	zulässige Dienste, siehe (29)	(13)

Erklärung. (1), (2): I (`I`) sind alle Mitarbeitenden der gewählten Stationen und ihrer Springerpools, U (`U`) alle Planungseinheiten. Dienste gibt es nur auf Stationen U^{St} (`dataset.station_ids`); Springerpools sind nur Herkunft. (3), (4): Jede Schicht hat eine Art. Kurz geschrieben sind S^F, S^S, S^N die Früh-, Spät- und Nachtschichten (`S_early`, `S_late`, `S_night`). Jede Regel gilt so für alle Schichten einer Art, unabhängig vom Kürzel. In der Demo hat jede Art genau eine Schicht: F, Z, S und N. (6), (8): D (`D`) ist der Monat. D^{-} (`D_prev`) sind die Vortage, für die die tatsächlichen Dienste bekannt sind: Ist der Dienstplan des Vormonats geladen, ist D^{-} der ganze Vormonat; sonst sind es die $m = 5$ Tage davor, an denen niemand gearbeitet hat. Der Folgetag $n+1$ in D^{+} zählt für Abwesenheiten, in die eine Nacht vom letzten Monatstag hineinreicht. (7): $\hat{D}$ (`D_plan`) sind die Tage mit Dienstvariablen. Endet der Monat an einem Samstag, gehört der Sonntag $n+1$ (`D_next`) dazu und es ist $\hat{n} = n+1$, sonst $\hat{n} = n$; siehe Abschnitt 6. Allgemein gilt: Alle Tagesbereiche, die in die Vortage reichen (nicht-positive d), sind mit D^{-} geschnitten, umfassen also nur bekannte Vortage. (9), (10): D^{Sa} (`D_s`) enthält die Samstage, deren Sonntag im Monat liegt oder mitgeplant wird, dazu den letzten Vortag, wenn er ein Samstag ist; W (`W`) jedes Wochenende, dessen Samstag oder Sonntag im Monat oder in den Vortagen liegt, durch seinen Samstag und aufsteigend. Damit gehört das letzte Wochenende vor dem Monat dazu, gleich an welchem Wochentag der Monat beginnt. (11): D^{end} (`D_end`) sind die Tage, für die ein Blockende in den Nachtregeln gebraucht wird. (12): K (Schlüssel von `R_min`) enthält nur Zeilen mit tatsächlichem Bedarf.

3 Parameter

3.1 Eingabedaten des Monats

$r_{sdul} \in \mathbb{N}$	Mindestbesetzung von Schicht s am Tag d auf Station u mit Qualifikation l	(14)
$h_s \in \mathbb{N}$	bezahlte Minuten der Schicht s	(15)
$t_i,\ c_i \in \mathbb{N}$	Sollminuten und gutgeschriebene Minuten im Monat	(16)
$\lambda_{iud} \in L$	Qualifikation der an Tag d gültigen Zugehörigkeit von i zu u	(17)
$y^0_{isd} \in \{0,1\},\ d \in D^-$	fester Dienst s von i an Vortag d	(18)
$T_i \subseteq D^+$	Fortbildungstage von i	(19)
$B \subseteq I \times D^+,\ V \subseteq B$	blockierte Tage, davon Urlaubstage	(20)
$\mathrm{home}(i,d) \in U$	Stammeinheit von i am Tag d	(21)

Abgeleitet aus dem Bedarf:

$$R_{usd} = \textstyle\sum_l r_{sdul}, \quad R_{ud} = \sum_s R_{usd}, \quad R_u = \sum_{d \in D} R_{ud} \tag{22}$$

$$U^D = \{u \in U^{St} : R_u > 0\} \tag{23}$$

$$\mathcal{B} = \{(u,s,l) : \exists\, d \in D \text{ mit } r_{sdul} > 0\} \tag{24}$$

$$I_{udl} = \{i \in I : \lambda_{iud} = l\} \tag{25}$$

Erklärung. (14) (`R_min`) stammt aus der Seite „Mindestbesetzung“. (15) (`h`) sind die Netto-Minuten der Schicht, die auf das Monatskonto gebucht werden. In der Demo: F 05:55–13:25 (420 min), Z 08:30–14:15 (345 min), S 13:15–21:00 (435 min), N 20:10–06:10 (555 min). (16) (`t`, `credits`): In der Demo ist das Tagessoll einer Vollzeitkraft 462 min (38,5 h je Woche) an jedem Werktag von Montag bis Freitag, der kein Feiertag ist; Teilzeitkräfte haben es anteilig. Für Urlaub, Fortbildung und Krankheit an diesen Tagen wird das Tagessoll gutgeschrieben, für freie Tage nicht. Krankheit ist in der Demo die Einschränkung „krank“ (intern `unavailable`). (17) (`level_at`): Ein Dienst zählt mit der Qualifikation der Stationszugehörigkeit, nicht mit der der Person. (18) (`past_shifts`) sind die Dienste des geladenen Vormonats; ohne Vormonat sind alle 0. (19) (`training`, `training_days`), (20) (`blocked`, `vacation`): B enthält alle Tage mit Urlaub, freiem Tag, Fortbildung oder „krank“, V nur die Urlaubstage. Alle drei umfassen auch den Folgetag $n+1$ (`context.availability`). (21) (`home_of`) ist die Einheit der an d gültigen Stammzugehörigkeit. (22)–(25): R_{usd}, R_{ud}, R_u (`required_usd`, `required_ud`, `required_u`) sind der Bedarf je Schicht, Tag und Monat über alle Qualifikationen. $\mathcal{B}$ (`demanded`) enthält die Kombinationen aus Station, Schicht und Qualifikation, die im Monat überhaupt Bedarf haben. I_{udl} (`staff_for`) sind die Personen, die an d auf u als l zählen.

3.2 Im Projekt gesetzte Parameter

Symbol	Code	Wert	Bedeutung
τ	`POLICY.balance_tolerance_minutes`	300	erlaubte Saldoabweichung in Minuten (5 h)
K^S	`POLICY.max_consecutive_work_days`	10	höchstens so viele Arbeitstage in Folge
N^{max}	`POLICY.max_consecutive_nights`	3	höchstens so viele Nächte in Folge
L^N	`L_n`	$\lfloor 2880/1440 \rfloor = 2$	ganz freie Tage nach einem Nachtblock
Δ^N	`Dist_n`	7	Tage zwischen Ende eines Nachtblocks und Beginn des nächsten
Δ^R	`Rot_w`	28	Tage eines Rotationsfensters
ω	`wish_budget`	840	Gewicht aller Wünsche einer Person zusammen
π	`transfer_penalty`	280	Strafe für einen Dienst auf fremder Station ($\omega/3$)

4 Zulässige Dienste

$$N^V = \{(i,s,d) : \text{für } (i,d) \text{ ist eine Einschränkung eingetragen, die } s \text{ nicht erlaubt}\} \tag{26}$$

$$N^N = \{(i,s,d) : s \in S^N,\ d \in D,\ (i,d+1) \in B\} \tag{27}$$

$$N^S = \{(i,s,d) : s \in S^S,\ d \in D,\ \big((i,d+1) \in V,\ (i,d) \notin V\big) \text{ oder } \big(d+1 \in T_i,\ d \notin T_i\big)\} \tag{28}$$

$$A = \big\{(i,s,d,u) : u \in U^{St},\ \lambda_{iud} \text{ definiert},\ (i,s,d) \notin N^V \cup N^N \cup N^S, \\ s \in S^F \cup S^S \text{ oder } (u,s,\lambda_{iud}) \in \mathcal{B}\big\} \tag{29}$$

Erklärung. Statt diese Regeln als Nebenbedingungen zu formulieren, werden verbotene Dienste gar nicht erst als Variable angelegt (`not_allowed`, Liste beim Anlegen von `assignment`). (26): Urlaub, freier Tag, Fortbildung und „krank“ verbieten alle Schichten, „nur bestimmte Schichten“ verbietet die übrigen. Für einen mitgeplanten Sonntag gelten die Einträge, die der Folgemonat für ihn schon hat. (27): Eine Nacht reicht in den Folgetag; vor einem blockierten Tag ist sie verboten, am letzten Monatstag also auch vor einer Abwesenheit am Ersten des Folgemonats. (28): Am Tag vor Urlaubsbeginn und vor einer Fortbildung gibt es keinen Spätdienst, auch wenn sie am Ersten des Folgemonats beginnen. Vor dem Urlaub ist das betriebliche Gepflogenheit; die Fortbildung ist Arbeitszeit, und nach einem Spätdienst bis 21:00 blieben 11 h Ruhezeit nur bis zu einem Beginn um 08:00, dessen Uhrzeit das Modell nicht kennt. (29) (`always_open`, `demanded`): Ein Dienst ist nur auf einer Station möglich, zu der an diesem Tag eine gültige Zugehörigkeit besteht. Früh- und Spätdienste kann dort jede Qualifikation übernehmen, auch ohne eigenen Bedarf; sie zählen dann als zusätzliches Personal. Nacht- und Zwischendienste dagegen nur, wenn die Station sie von dieser Qualifikation im Monat überhaupt fordert (z. B. kein Zwischendienst, wenn nie einer gefordert ist, keine Nächte für Azubis, wenn nie Azubi-Nachtbedarf besteht).

5 Variablen und Hilfsausdrücke

5.1 Entscheidungsvariablen

$x_{isdu} \in \{0,1\}$	$(i,s,d,u) \in A$	Dienst (`assignment`)	(30)
$\psi_{iw\theta} \in \{0,1\}$	$(i,w,\theta) \in \Psi$	Schichtart im Rotationsfenster (`rotation`)	(31)
$g_{sdul} \geq 0$	$(s,d,u,l) \in K$	Lücke (`gap`)	(32)
$\eta_i \in \mathbb{R}$	$i \in I$	Monatssaldo in Minuten (`hour`)	(33)
$o_i \geq 0$	$i \in I$	Saldo außerhalb des Bands (`over`)	(34)
$q_{ud} \geq 0$	$u \in U^{St},\ d \in D,\ R_{ud} > 0$	Quadrat der Lücken des Tages (`gap_sq`)	(35)
$\rho_{ud} \geq 0$	$u \in U^{D},\ d \in D$	Abweichung der Tagesbesetzung (`day_dev`)	(36)
$\chi_i \geq 0$	$i \in I$	Quadrat der gearbeiteten Samstage (`sat_sq`)	(37)
$\delta_{usd} \geq 0$	$u \in U^{St},\ d \in D,\ R_{ud} > 0,\ s \in S$	Abweichung der Schichtbesetzung (`dev`)	(38)
$b_{id} \in [0,1]$	$i \in I,\ d \in \{0\} \cup D$	Beginn eines Nachtblocks (`n_beg`)	(39)
$e_{id} \in [0,1]$	$i \in I,\ d \in D^{end}$	Ende eines Nachtblocks (`n_end`)	(40)
$z_{id} \in [0,1]$	$i \in I,\ d \in D$	Spät- vor Frühdienst (`L_to_E`)	(41)
$v_{isd} \in [0,1]$	$i \in I,\ s \in S,\ d \in D$	gleiche Schicht wie am Vortag (`cons_s`)	(42)

Erklärung. Die eigentliche Entscheidung ist x (30). ψ (31) ist ebenfalls binär; die Menge Ψ steht bei der Rotation in Abschnitt 7.8. Alle anderen Variablen sind stetige Hilfsgrößen; ihre Werte ergeben sich aus x und der Optimierung. g (32) nimmt im Optimum ganzzahlige Werte an, b, e, z und v (39)–(42) nur 0 oder 1, weil sie ausschließlich durch ganzzahlige Ausdrücke in x nach oben bzw. unten begrenzt werden und die Optimierung sie an diese Grenze drückt.

5.2 Hilfsausdrücke

$$X_{isd} = \begin{cases} \sum_{u:(i,s,d,u)\in A} x_{isdu} & d \in \hat{D} \\ y^0_{isd} & d \in D^- \\ 0 & \text{sonst} \end{cases} \qquad (\texttt{x}, \texttt{shift_on}) \qquad (43)$$

$$Y_{id} = \sum_{s\in S} X_{isd} \qquad (\texttt{work}) \qquad (44)$$

$$Y^F_{id} = \sum_{s\in S^F} X_{isd}, \quad Y^S_{id} = \sum_{s\in S^S} X_{isd}, \quad Y^N_{id} = \sum_{s\in S^N} X_{isd} \qquad (\texttt{early}, \texttt{late}, \texttt{night})$$

$$P_{usd} = \sum_{i:(i,s,d,u)\in A} x_{isdu} \qquad (\texttt{staffed}) \qquad (45)$$

$$P_{ud} = \sum_s P_{usd}, \quad P_u = \sum_{d\in D} P_{ud} \qquad (\texttt{staff_of_day}, \texttt{staff_of_month})$$

$$G_{ud} = \sum_{(s,d,u,l)\in K} g_{sdul} \qquad (\texttt{gaps_of_day}) \qquad (46)$$

Erklärung. (43): X_{isd} ist 1, wenn i an d Schicht s arbeitet, egal auf welcher Station. Auf den Vortagen ist es der feste Dienst aus dem Vormonat, außerhalb des bekannten Zeitraums 0. Dadurch wirken alle Regeln über den Monatswechsel, ohne dass die Vergangenheit verändert werden kann. (44): Y_{id} ist 1, wenn i an d irgendeinen Dienst hat, Y^F, Y^S, Y^N entsprechend für Früh-, Spät- und Nachtdienst. (45): Besetzung einer Station je Schicht, je Tag und im Monat. (46): Summe der Lücken einer Station an einem Tag.

6 Feste Regeln

Diese Nebenbedingungen hält jeder Plan ein. Zusammen mit den verbotenen Diensten aus (29) sind das alle festen Regeln. Am Monatsanfang erzwingen sie nie etwas, das die Vergangenheit unmöglich machen könnte: So bleibt das Modell auch mit geladenem Vormonat immer lösbar.

Der mitgeplante Sonntag. Die Regeln gelten auf allen Planungstagen $\hat{D}$. Endet der Monat an einem Samstag, ist das auch der Sonntag $n+1$ (`D_next`). Er ist durch (57) eine Kopie des Samstags: dieselbe Schicht oder beide frei. So plant der Monat nur Samstagsdienste, die am Sonntag weitergehen können, ohne eine vierte Nacht, gegen die Erholung oder eine Abwesenheit am Sonntag. Der Sonntag zählt aber für kein Konto (61), keinen Bedarf und kein Ziel, und seine Dienste gehören nicht zum Plan. Der Folgemonat plant ihn mit diesem Plan als Vormonat neu und führt das Wochenende über (64) fort.

6.1 Ein Dienst pro Tag und Arbeitstage am Stück

$$Y_{id} \le 1 \qquad \forall i \in I,\ d \in \hat{D} \tag{47}$$

$$\sum_{d'=d}^{d+K^S} Y_{id'} + |T_i \cap \{d, \dots, d+K^S\}| \le \max\big(K^S,\ \kappa_{id}\big) \qquad \forall i \in I,\ d \in \{1 - \min(m, K^S), \dots, \hat{n} - K^S\} \tag{48}$$

Erklärung. (47) (`one_shift`): Höchstens ein Dienst pro Person und Tag, über alle Stationen. (48): In jedem Fenster von $K^S + 1 = 11$ Tagen höchstens $K^S = 10$ Arbeitstage; Fortbildungstage zählen als Arbeitstage. Die Fenster beginnen schon in den Vortagen, damit die Regel über den Monatswechsel gilt. κ_{id} ist der feste Teil des Fensters (Arbeitstage der Vortage und Fortbildungstage). Erreicht er die Grenze schon allein, ist im Fenster kein weiterer Dienst möglich, statt dass das Modell unlösbar wird.

6.2 Nachtdienste

$$\sum_{d'=d}^{d+N^{max}} Y^N_{id'} \le \max\big(N^{max},\ \nu_{id}\big) \qquad \forall i \in I,\ d \in \{1 - \min(m, N^{max}), \dots, 0\} \cup \{1, \dots, \hat{n} - N^{max}\} \tag{49}$$

$$b_{id} \ge Y^N_{id} - Y^N_{i,d-1} \qquad \forall i \in I,\ d \in \{0\} \cup \hat{D} \tag{50}$$

$$e_{id} \ge Y^N_{id} - Y^N_{i,d+1} \qquad \forall i \in I,\ d \in D^{end} \tag{51}$$

$$b_{id} + e_{id} \le 1 \qquad \forall i \in I,\ d \in \{1, \dots, \hat{n} - 1\} \tag{52}$$

$$e_{id} + Y_{i,d+j} \le 1 \qquad \forall i \in I,\ d \in \{-L^N, \dots, \hat{n} - 1\},\ j = 1, \dots, L^N,\ d + j \in \hat{D} \tag{53}$$

$$e_{id} \le 0 \qquad \forall i \in I,\ d \in \{1, \dots, \hat{n} - 1\} \text{ mit } T_i \cap \{d+1, \dots, d+L^N\} \ne \emptyset \tag{54}$$

$$e_{id} + Y^F_{i,d+L^N+1} \le 1 \qquad \forall i \in I,\ d \in \{-L^N, \dots, \hat{n} - 1\},\ d + L^N + 1 \in \hat{D} \tag{55}$$

$$e_{id} + \sum_{j=1,\ d+j \in \hat{D}}^{\Delta^N} b_{i,d+j} \le 1 \qquad \forall i \in I,\ d \in \{1 - \Delta^N, \dots, \hat{n} - 1\} \cap D^{end} \tag{56}$$

Erklärung. (49): In jedem Fenster von $N^{max} + 1 = 4$ Tagen höchstens $N^{max} = 3$ Nächte; ν_{id} sind die Nächte der Vortage im Fenster (wie bei (48)). (50), (51) (`n_beg`, `n_end`): b_{id} ist mindestens 1, wenn an d ein Nachtblock beginnt (Nacht an d, keine an $d-1$), e_{id}, wenn einer endet. Untergrenzen genügen, weil b und e nur in $\le$-Bedingungen mit positivem Vorzeichen und in keinem Ziel vorkommen. (52): Beginn und Ende am selben Tag hieße Ein-Nacht-Block; der ist im Monat verboten. Eine einzelne Nacht am letzten Vortag wird nicht zwangsweise fortgesetzt, weil eine Abwesenheit am Ersten oder die Nachtgrenze die zweite Nacht verbieten kann. (53), (54), (55): Nach dem letzten Nachtdienst eines Blocks sind $L^N = 2$ Tage ganz frei, und am dritten Tag gibt es keinen Frühdienst. Das bildet die 48 Stunden Erholung ab: Die Nacht endet um 06:10, der Frühdienst am dritten Tag beginnt um 05:55. *Ganz frei* heißt auch ohne

Fortbildung (54) (`recovery_training`): Ein Block im Monat endet nicht innerhalb von L^N Tagen vor einer Fortbildung, auch nicht vor einer am Folgetag $n+1$. Mit (27) gibt es so in den L^N Tagen vor einer Fortbildung gar keine Nacht. Nur Tage im Monat werden eingeschränkt; ein Block, der schon vor dem Monat geendet hat, bleibt, wie er ist. (56): Nach dem Ende eines Nachtblocks darf in den nächsten $\Delta^N = 7$ Tagen kein neuer Block beginnen; berücksichtigt werden nur Blockanfänge im Monat.

6.3 Wochenenden

$$X_{isd} = X_{is,d+1} \qquad \forall i \in I,\ s \in S,\ d \in D^{Sa} \cap D \tag{57}$$

$$X_{is1} \le X_{is0} \qquad \forall i \in I,\ s \in S,\ \text{falls } 0 \in D^{Sa},\ Y_{i0} = 1 \tag{58}$$

$$\Omega_{iw} = \begin{cases} \max\big(Y_{iw},\, Y_{i,w+1}\big) & w \le 0 \\ Y_{iw} & w \ge 1 \end{cases} \qquad (\texttt{weekend}) \tag{59}$$

$$\Omega_{i,w_k} + \Omega_{i,w_{k+1}} \le 1 \qquad \forall i \in I,\ k = 1, \dots, |W| - 1 \text{ mit } w_{k+1} \ge 0 \tag{60}$$

Erklärung. (57): Samstag und Sonntag haben dieselbe Schicht, oder beide sind frei. (58): Über den Monatswechsel (der Monat beginnt an einem Sonntag) bindet der bekannte Samstag den Sonntag als feste Regel nur schwach: Nach einem gearbeiteten Samstag hat der Sonntag dieselbe Schicht oder ist frei; erzwungen wird nichts, weil eine Abwesenheit am Ersten oder die Nachtgrenze den Dienst verbieten kann. War der Samstag frei, ist der Sonntag offen; ohne Vormonat gelten die Vortage ja nur als frei, sonst müsste jeder am Ersten frei haben. Mit geladenem Vormonat verlangt das Ziel (64) die volle Regel. (59): Ω_{iw} gibt an, ob i das Wochenende mit dem Samstag w arbeitet. Gearbeitet ist es mit einem Dienst am Samstag oder Sonntag; eine Nacht vom Freitag allein lässt es frei. Im Monat haben Samstag und Sonntag dieselbe Schicht (57), also entscheidet der Samstag ($w \ge 1$). Für $w \le 0$ liegt der Samstag vor dem Monat und Y_{iw} fest (außerhalb der bekannten Tage 0); das Maximum ist dann linear: 1 nach einem gearbeiteten Samstag, sonst $Y_{i,w+1}$. (60): Keine zwei Wochenenden in Folge, auch nicht über den Monatswechsel. Geprüft werden nur Paare, deren späteres Wochenende den Monat berührt ($w_{k+1} \ge 0$, sein Sonntag liegt also im Monat); die Vergangenheit wird nie gegen sich selbst geprüft, auch nicht ein Paar, das nach einem gearbeiteten Samstag vor dem Monat schon ganz feststeht. Daraus folgt zugleich, dass höchstens jedes zweite Wochenende gearbeitet wird.

7 Ziele in ihrer Rangfolge

Jedes Ziel steht hier mit den Nebenbedingungen, die seine Hilfsvariablen festlegen. Die Reihenfolge ist die Rangfolge, in der Gurobi sie optimiert (Abschnitt 8).

7.1 Rang 1: Monatssaldo innerhalb des Bands

$$\textstyle\sum_{(s,d,u):(i,s,d,u)\in A,\ d\in D} h_s\, x_{isdu} + c_i - \eta_i = t_i \qquad \forall i \in I \tag{61}$$

$$o_i \ge \eta_i - \tau, \qquad o_i \ge -\eta_i - \tau \qquad \forall i \in I \tag{62}$$

$$f_1 = \textstyle\sum_{i\in I} o_i \to \min \qquad (\texttt{balance_violation}) \tag{63}$$

Erklärung. (61) (`workhours`): η_i ist der Saldo in Minuten: geplante Minuten im Monat plus Gutschriften minus Soll, genau wie das Monatskonto im Projekt. Ein mitgeplanter Sonntag zählt nicht mit. (62) (`over_up`, `over_down`): o_i misst, um wie viele Minuten der Saldo außerhalb von $\pm\tau = \pm 300$ liegt; im Optimum ist $o_i = \max(0, |\eta_i| - \tau)$. (63): Dieses Ziel steht ganz oben, noch vor den Lücken. Können alle

Salden im Band liegen, ist $\hat{f}_1 = 0$, und jede spätere Stufe muss $f_1 = 0$ halten (87): Das Band wirkt dann wie eine feste Regel, auch wenn dafür eine Lücke offen bleibt. Ist es für jemanden unerreichbar (z. B. den halben Monat frei, freie Tage werden nicht gutgeschrieben), entsteht trotzdem ein Plan mit möglichst kleiner Überschreitung, statt dass das Modell unlösbar wird.

7.2 Rang 2: Wochenende über den Monatswechsel

$$f_2 = \sum_{i \in I : Y_{i0}=1} \left(1 - X_{i\sigma_i 1}\right) + \sum_{i \in I : Y_{i0}=0} Y_{i1} \to \min \qquad (\texttt{weekend_start}) \tag{64}$$

nur mit geladenem Vormonat und wenn der Monat an einem Sonntag beginnt; sonst ist $f_2 = 0$. Dabei ist σ_i die Schicht von i am Samstag 0.

Erklärung. (64) (`weekend_split`): Zählt die Personen, deren Wochenende über den Monatswechsel ungleich ist: Wer Samstag gearbeitet hat und am Sonntag nicht dieselbe Schicht hat, und wer Samstag frei hatte und am Sonntag arbeitet. Andere Schichten als die vom Samstag schließt (58) schon aus. Weil das Ziel direkt nach dem Saldo kommt, gilt die volle Wochenendregel, wann immer das möglich ist. Ist es nicht möglich, etwa weil nach dem Plan des Vormonats noch Urlaub am Ersten eingetragen wurde, bleibt das Wochenende geteilt, statt dass das Modell unlösbar wird; die Prüfung weist es dann aus. Hat der Vormonat mit seinem mitgeplanten Sonntag gerechnet, kann jeder Samstagsdienst weitergehen. Ohne Vormonat gilt dieses Ziel nicht, weil die Tage davor nur als frei angenommen sind.

7.3 Rang 3: Lücken

$$\sum_{i \in I_{udl} : (i,s,d,u) \in A} x_{isdu} + g_{sdul} \geq r_{sdul} \qquad \forall (s,d,u,l) \in K \tag{65}$$

$$f_3 = \sum_{(s,d,u,l) \in K} g_{sdul} \to \min \qquad (\texttt{gaps}) \tag{66}$$

Erklärung. (65) (`demand`): Die Mindestbesetzung jeder Bedarfszeile wird durch Dienste mit passender Qualifikation gedeckt; was fehlt, ist eine Lücke g. Die Mindestbesetzung ist damit keine feste Regel: Auch bei Personalmangel gibt es einen Plan, und (66) hält die Zahl der unbesetzten Pflichtstellen klein.

7.4 Rang 4: Lücken gleichmäßig verteilen

$$q_{ud} \geq (2k+1)\, G_{ud} - k(k+1) \qquad \forall (u,d) : R_{ud} > 0,\ k = 0, \ldots, R_{ud} - 1 \tag{67}$$

$$f_4 = \sum_{(u,d) : R_{ud} > 0} \frac{q_{ud}}{R_{ud}} \to \min \qquad (\texttt{gap_spread}) \tag{68}$$

Erklärung. (67) (`gap_sq`): Lineare Darstellung von $q_{ud} \geq G_{ud}^2$. Die rechte Seite ist für jedes k die Sekante durch die Parabelpunkte (k, k^2) und $(k+1, (k+1)^2)$. Zwischen k und $k+1$ liegt sie über der Parabel, außerhalb darunter. Das Maximum aller Sekanten ist daher die stückweise lineare Interpolation von G_{ud}^2: überall mindestens G_{ud}^2 und in ganzzahligen Punkten genau G_{ud}^2. Da die Lücken eines Tages ganzzahlig zwischen 0 und R_{ud} liegen und q_{ud} minimiert wird, ist $q_{ud} = G_{ud}^2$ im Optimum exakt. (68): Durch das Quadrat sind zwei Tage mit je einer Lücke $(1+1)$ billiger als ein Tag mit zwei (4). Die Division durch R_{ud} gewichtet relativ zum Bedarf: Eine Lücke wiegt an einem schwach besetzten Tag schwerer.

7.5 Rang 5: Tage gleichmäßig besetzen

$$\rho_{ud} \geq P_{ud} - \tfrac{R_{ud}}{R_u} P_u, \qquad \rho_{ud} \geq -\big(P_{ud} - \tfrac{R_{ud}}{R_u} P_u\big) \qquad \forall u \in U^D,\ d \in D \tag{69}$$

$$f_5 = \textstyle\sum_{u \in U^D} \sum_{d \in D} \rho_{ud} \to \min \qquad (\texttt{staffing_spread}) \tag{70}$$

Erklärung. (69) (`day_dev_up`, `day_dev_down`): $\rho_{ud} \geq |P_{ud} - \frac{R_{ud}}{R_u} P_u|$, also die Abweichung der Tagesbesetzung von ihrem Anteil an der Monatsbesetzung. Ein Tag ohne Bedarf hat Anteil 0, jeder Dienst dort zählt als Abweichung. (70): Jeder Tag einer Station soll im gleichen Verhältnis Ist/Soll besetzt sein wie der ganze Monat; Über- und Unterbesetzung verteilen sich so gleichmäßig auf die Tage.

7.6 Rang 6: Samstage gleichmäßig verteilen

$$a_i = \textstyle\sum_{d \in D,\ d \text{ Samstag}} Y_{id} \qquad \forall i \in I \tag{71}$$

$$\chi_i \geq (2k+1)\, a_i - k(k+1) \qquad \forall i \in I,\ k = 0, \ldots, s^{Sa} - 1 \tag{72}$$

$$f_6 = \textstyle\sum_{i \in I} \chi_i \to \min \qquad (\texttt{saturdays}) \tag{73}$$

Erklärung. (71) (`saturdays_of`): a_i ist die Zahl der Samstage im Monat, an denen i arbeitet; s^{Sa} ist die Zahl der Samstage im Monat. Wegen (57) ist das zugleich die Zahl der gearbeiteten Wochenenden. (72) (`sat_sq`): $\chi_i \geq a_i^2$, mit denselben Sekanten wie bei (67); im Optimum ist $\chi_i = a_i^2$. (73): Bei gegebener Zahl gearbeiteter Samstage ist die Summe der Quadrate am kleinsten, wenn sich die Zahlen höchstens um eins unterscheiden: $2 + 2 + 3$ ist besser als $1 + 3 + 3$. Jede Qualifikation deckt ihren eigenen Bedarf, deshalb gleicht die Summe über alle Personen die Samstage innerhalb jeder Qualifikation aus. Wer an einem Samstag nicht kann (Urlaub, Einschränkung), hat entsprechend weniger; die übrigen teilen den Rest gleichmäßig. Die Zahlen sind absolut, also unabhängig von Teilzeit. Durch die Sekanten ist die Schranke der Relaxation in der Regel schon der ganzzahlige Wert, sodass Gurobi die Stufe schnell exakt löst.

7.7 Rang 7: Wünsche

$$\omega_i = \text{round}(\omega / n_i), \qquad n_i = \text{Zahl der Wünsche von } i \text{ im Monat} \tag{74}$$

$$\begin{aligned} p_{isdu} = {} & -\pi \cdot \mathbf{1}\big[\text{home}(i,d) \in U^{St},\ \text{home}(i,d) \neq u\big] \\ & - \omega_i \cdot \mathbf{1}[\text{Wunsch „freier Tag“ für } (i,d)] - \omega_i \cdot \mathbf{1}\big[s \in S^N,\ \text{Wunsch „freier Tag“ für } (i,d+1)\big] \\ & - \omega_i \cdot \mathbf{1}[\text{Wunsch „Schicht } s \text{ frei“ für } (i,d)] \\ & + \omega_i \cdot \mathbf{1}[\text{Wunsch „Wunschtag“ für } (i,d)] + \omega_i \cdot \mathbf{1}[\text{Wunsch „Wunschschicht } s\text{“ für } (i,d)] \end{aligned} \tag{75}$$

$$f_7 = \textstyle\sum_{(i,s,d,u) \in A} p_{isdu}\, x_{isdu} \to \max \qquad (\texttt{wishes}) \tag{76}$$

Erklärung. (74) (`wish_reward`): Jede Person hat für alle ihre Wünsche des Monats zusammen dasselbe Gewicht $\omega = 840$, und jeder ihrer n_i Wünsche zählt ω / n_i. Wer viele Wünsche einträgt, bekommt also pro Wunsch weniger und gewinnt dadurch keinen größeren Einfluss auf den Plan: Der einzige Wunsch einer Person wiegt so viel wie alle zehn Wünsche einer anderen zusammen. 840 ist durch 1 bis 8 teilbar; ab neun Wünschen wird auf ganze Zahlen gerundet, damit die Stufe ganzzahlige Werte behält (Abschnitt 8). (75) (`pref`): Jeder zulässige Dienst bekommt einen Präferenzwert. Ein erfüllter Dienstwunsch bringt $+\omega_i$, ein Dienst gegen einen Freiwunsch kostet ω_i. Beim freien Tag zählt auch ein Nachtdienst am Vorabend, weil er bis 06:10 in den Wunschtag hineinreicht; so bewertet es auch die Prüfung. Ein Dienst auf einer fremden Station kostet π, ein Drittel des einzigen Wunsches einer Person, aber nur, wenn die Person eine

Station als Stammeinheit hat; Springer (Stammeinheit ist ein Pool) dürfen überall gleich gut arbeiten. In der Demo mit nur einer Station kommt das nicht vor. (76): Die Summe der Präferenzwerte aller Dienste wird maximiert.

7.8 Rang 8: Rotation

$$\mathcal{W} = \{w \in \bar{D} : 1 \le w + \Delta^R - 1 \le n\}, \qquad [w] = \{w, \dots, w + \Delta^R - 1\} \tag{77}$$

$$\Psi = \big\{(i, w, \theta) : w \in \mathcal{W},\ \ \theta \in \Theta \setminus \{\text{Zwischen}\},\ y^0_{isd} = 0\ \forall s \in S^\theta,\ d \in [w] \cap D^-,\ \exists\,(i, s, d, u) \in A \text{ mit } s \in S^\theta,\ d \in [w] \cap D\big\} \tag{78}$$

$$\psi_{iw\theta} \le \sum_{d \in [w] \cap D} \sum_{s \in S^\theta} X_{isd} \qquad \forall (i, w, \theta) \in \Psi \tag{79}$$

$$f_8 = \sum_{(i,w,\theta) \in \Psi} \psi_{iw\theta} \to \max \qquad (\texttt{rotation}) \tag{80}$$

Erklärung. Niemand soll wochenlang nur dieselbe Schicht haben: In jedem Fenster von $\Delta^R = 28$ Tagen soll jede Person möglichst jede Schichtart haben, die sie arbeiten kann. Gezählt werden nur Früh, Spät und Nacht; der Zwischendienst ist ein kurzer Zusatzdienst, in den niemand der Abwechslung wegen gedrängt werden soll. (77) (`window_starts`): $\mathcal{W}$ sind die ersten Tage aller Fenster, die im Monat enden und an einem bekannten Tag beginnen. Mit geladenem Vormonat reichen sie bis zu 27 Tage in ihn zurück, sodass die Rotation über den Monatswechsel weiterläuft; ohne ihn beginnen sie frühestens $m = 5$ Tage vor dem Monat (ein Monat mit 31 Tagen hat dann 9 Fenster). (78) (Schlüssel von `worked_type`): Gezählt wird ein Paar aus Fenster und Schichtart nur, wenn die Person diese Art im Fensterteil des Monats überhaupt arbeiten kann, also eine Variable dafür existiert. Früh und Spät kann jede Qualifikation (29); Azubis werden aber nicht in Nächte gedrängt, wenn nie Azubi-Nachtbedarf besteht, und „nur bestimmte Schichten“ lässt die übrigen Arten aus. Hat die Person die Art im Fensterteil vor dem Monat schon gearbeitet, ist das Paar erfüllt und braucht keine Variable. (79): $\psi_{iw\theta}$ kann nur 1 sein, wenn i im Fenster ab w mindestens eine Schicht der Art θ arbeitet. (80): Die Zahl der erfüllten Paare aus Person, Fenster und Schichtart wird maximiert. Ohne Vormonat überdecken alle Fenster die Monatsmitte; dann genügt eine Schicht jeder Art dort für alle Fenster.

7.9 Rang 9: Schichten eines Tages gleichmäßig besetzen

$$\delta_{usd} \ge P_{usd} - \tfrac{R_{usd}}{R_{ud}} P_{ud}, \qquad \delta_{usd} \ge -\big(P_{usd} - \tfrac{R_{usd}}{R_{ud}} P_{ud}\big) \qquad \forall (u, d) : R_{ud} > 0,\ s \in S \tag{81}$$

$$f_9 = \sum_{u,s,d} \delta_{usd} \to \min \qquad (\texttt{shift_balance}) \tag{82}$$

Erklärung. (81) (`shift_dev_up`, `shift_dev_down`): $\delta_{usd} \ge |P_{usd} - \frac{R_{usd}}{R_{ud}} P_{ud}|$, die Abweichung einer Schicht von ihrem Anteil an der Tagesbesetzung. (82): Zusätzliches Personal eines Tages verteilt sich im Verhältnis der Mindestbesetzung auf die Schichten. Weil sich Anteile nur mit „Bruchteilen von Personen“ exakt treffen lassen, ist ein Teil dieser Abweichung unvermeidbar; die Stufe hat deshalb ein Zeitlimit statt eines Optimalitätsbeweises.

7.10 Rang 10: Wechsel vom Spät- in den Frühdienst

$$z_{id} \ge Y^S_{i,d-1} + Y^F_{id} - 1 \qquad \forall i \in I,\ d \in D \tag{83}$$

$$f_{10} = \sum_{i \in I} \sum_{d \in D} z_{id} \to \min \qquad (\texttt{Min_L_to_E}) \tag{84}$$

Erklärung. (83) (`L_to_E`): z_{id} ist mindestens 1, wenn auf einen Spätdienst am Vortag ein Frühdienst folgt, auch über den Monatswechsel. (84): Zwischen Spätdienst-Ende (21:00) und Frühdienst-Beginn (05:55) liegen nur 8:55 h. Mit den Schichtzeiten der Demo und den Nachtregeln ist das der einzige mögliche Fall unter 11 h Ruhezeit. Er ist bewusst erlaubt, wird aber so selten wie möglich geplant.

7.11 Rang 11: Dieselbe Schicht wie am Vortag

$$v_{isd} \le X_{isd}, \qquad v_{isd} \le X_{is,d-1} \qquad \forall i \in I,\ s \in S,\ d \in D \tag{85}$$

$$f_{11} = \sum_{i \in I} \sum_{s \in S} \sum_{d \in D} v_{isd} \to \max \qquad (\texttt{cons}) \tag{86}$$

Erklärung. (85) (`cons_s`): v_{isd} darf nur 1 sein, wenn an d und $d-1$ dieselbe Schicht gearbeitet wird. (86): Möglichst oft dieselbe Schicht wie am Vortag, damit ruhige Folgen entstehen.

8 Lexikografisches Lösen

Für $k = 1, \dots, 11$ wird f_k über alle zulässigen Lösungen optimiert, unter den zusätzlichen Bedingungen

$$f_j \le \hat{f}_j + \alpha_j \quad (\text{bzw. } f_j \ge \hat{f}_j - \alpha_j \text{ für die Maximierungen } j = 7, 8, 11) \qquad \forall j < k, \tag{87}$$

wobei $\hat{f}_j$ der in Stufe j erreichte Wert ist. Gurobi setzt das als Mehrzieloptimierung mit fallenden Prioritäten um (`setObjectiveN`). Die Toleranzen α_j (`abstol`) sind:

Stufe	Gesamtspielraum ε_j (`stage_slack`)	$\alpha_j = \varepsilon_j/2$
f_4 `gap_spread`	2	1
f_5 `staffing_spread`	$\lvert U^D \rvert \cdot n/7$	$\lvert U^D \rvert \cdot n/14$
f_9 `shift_balance`	$2\,\lvert U^D \rvert \cdot n/7$	$\lvert U^D \rvert \cdot n/7$
alle anderen (ganzzahlige Werte)	–	0,5

Erklärung. Eine spätere Stufe darf eine frühere nur um deren Toleranz verschlechtern. Bei den drei Verteilungsstufen f_4, f_5 und f_9 ist das bewusst etwas Spielraum, damit die späteren Stufen noch Wahlmöglichkeiten haben; die andere Hälfte des Spielraums erlaubt einen frühen Abbruch der Stufe (siehe unten). Insgesamt liegt der Endwert einer solchen Stufe höchstens ε_j über dem bestmöglichen Wert. Alle anderen Stufen haben ganzzahlige Werte (Minuten, Lücken, Anzahlen, Quadrate, Wunschpunkte) und halten ihren Bestwert exakt; die Toleranz 0,5 fängt nur Rundungsfehler ab. Mit einer winzigen Toleranz wie 10^{-6} würde ein als 294,000004 gemeldeter Bestwert jeden ganzzahligen Plan mit 294 ausschließen, und die folgenden Stufen fänden keinen anderen Plan mehr.

Abbruchkriterien.

Kriterium	Wert
Gesamtzeit T (`SOLVER_MAX_TIME_SECONDS`)	in der Demo fest 30 s
relative Optimalitätslücke (`MIPGap`)	3 %; exakt (0) für f_1, f_2, f_3, f_6 und f_8
absolute Optimalitätslücke (`MIPGapAbs`)	$\varepsilon_j/2$ für f_4, f_5, f_9
Zeitlimit einzelner Stufen (`stage_time_share`)	0,15 T für f_5 und f_7, 0,10 T für f_6, f_8, f_9
Watchdog	beendet die Rechnung spätestens 5 s nach T

Erklärung. f_1 und f_3 werden exakt gelöst: 3 % eines großen Werts (ein unerreichbares Konto oder viele Lücken) würden weitere Konten außerhalb des Bands oder vermeidbare Lücken zulassen; ebenso die kleine Zahl f_2 der geteilten Wochenenden am Monatsanfang. Auch f_6 wird exakt gelöst, weil schon ein Samstag mehr oder weniger zählt, und f_8, weil der Wert in die Hunderte geht und 3 % davon eine Person in vielen Fenstern ohne Nachtdienst lassen könnten. Die Zeitlimits schützen die folgenden Stufen: Bei f_5, f_6, f_8 und f_9 kann die Schranke den erreichbaren Wert verfehlen (Bruchteile von Personen, durch

Abwesenheiten oder die Wochenendregeln unerreichbare Werte), sodass Gurobi die Stufe sonst bis zur Gesamtzeit weiterrechnen würde. Bei den Wünschen f_7 schließt sich die Lücke mit vielen Wünschen nur langsam; ohne Zeitlimit kämen Rotation, Schichtverteilung, Spät→Früh und Folgen dann gar nicht an die Reihe. Die übrigen Stufen laufen, bis sie bewiesen sind oder T erreicht ist. Ist T erreicht, gilt der beste bis dahin gefundene Plan; spätere Stufen kommen dann nicht mehr an die Reihe.

9 Nach dem Lösen

Ergebnis. Aus der Lösung werden die Dienste $\{(i, s, d, u) : x_{isdu} = 1\}$ mit der Qualifikation λ_{iud} übernommen, nur für Tage des Monats: Ein mitgeplanter Sonntag gehört nicht zum Plan. Die gemeldeten Lücken werden nicht aus g gelesen, sondern aus den Diensten berechnet, $\max(0, r_{sdul} - \sum_{i \in I_{udl}} x_{isdu})$, ebenso die angezeigten Werte von f_3, f_4, f_5, f_6, f_8 und f_9. So stimmen sie genau mit der unabhängigen Prüfung überein, auch wenn Hilfsvariablen wegen der Toleranzen (87) über ihrem echten Wert liegen.

Unabhängige Prüfung. Die Prüfung (`api/app/domain/acceptance.py`) übernimmt vom Solver nichts außer den Diensten. Sie rechnet mit den echten Uhrzeiten und den Diensten des Vormonats und prüft unter anderem: Mindestbesetzung und gemeldete Lücken, Zugehörigkeit und Qualifikation, Einschränkungen samt keiner Nacht vor einem Abwesenheitstag und keinem Spätdienst vor Urlaubsbeginn oder Fortbildung (27), (28), einen Dienst pro Tag, den Monatssaldo innerhalb $\pm\tau$, tägliche Höchstarbeitszeit, Pausen und den Durchschnitt von 8 h je Werktag, 11 h Ruhezeit, höchstens N^{max} Nächte in Folge und 48 h Erholung nach der letzten Nacht. Dazu kommen die Regeln des Hauses: höchstens K^S Arbeitstage am Stück (Fortbildung zählt mit), am Wochenende dieselbe Schicht an beiden Tagen und nie zwei gearbeitete Wochenenden in Folge, jeweils auch über den Monatswechsel, sofern der Vormonat geladen ist. Ein Wochenende über den Monatswechsel, das (64) nicht fortführen konnte, wird also abgelehnt. Abweichend vom Gesetzestext gilt in der Demo-Version:

- **Spät → Früh** am Folgetag ist von der 11-h-Ruhezeit ausgenommen. Das Modell hält diese Wechsel über (84) klein, die Prüfseite zählt sie als Kennzahl. Jede andere Ruhezeit unter 11 h wird abgelehnt.
- **Erholung nach Nächten**: Auch eine Fortbildung an einem der L^N Tage nach der letzten Nacht ist ein Verstoß, passend zu (54). Ein Nachtdienst-Wunsch in den L^N Tagen vor einer Fortbildung gilt deshalb als nicht erfüllbar, ebenso nach (28) ein Spätdienst-Wunsch am Tag davor.
- **Zeitumstellung**: Modell und Prüfung ignorieren sie. Eine Schicht zählt für die tägliche Höchstarbeitszeit und die Pausen mit ihren geplanten Zeiten, auch die Nacht der Umstellung im Oktober, die tatsächlich eine Stunde länger dauert.
- **Ersatzruhetag**: Ob jeder gearbeitete Sonn- oder Feiertag durch einen freien Werktag ausgeglichen wird (ArbZG §11 Abs. 3), prüft die Demo-Version nicht; das Modell bildet diese Regel ebenfalls nicht ab.